Saint Joseph University
Faculty of Engineering
Master in Cybersecurity, INCI, Semester 3
Academic Year 2026-2027

Ethical Hacking

AI Security Orchestrator: HexStrike AI + Local LLM

Install, run a local LLM, connect it over MCP, and use it to test a lab network

Prepared by
Joe Haddad
Georges EL Hajj

November 2026

1. Introduction

The goal of this lab was to install an AI security orchestrator called HexStrike AI, run a local language model next to it so that nothing leaves our machine, connect the two together, and then use the setup to test a lab network. HexStrike AI is an MCP server that lets an AI agent drive more than 150 security tools (nmap, nikto, sqlmap, gobuster and many others) instead of us running each tool by hand.

The whole setup runs on our own hardware. The host is a Windows 11 laptop with an Intel i7-13700HX, 24 GB of RAM and an RTX 5050 laptop GPU with 8 GB of VRAM. On top of Windows we run Ollama, which serves the local model. Inside VirtualBox we run a Kali Linux virtual machine that holds HexStrike AI, and a Metasploitable 3 virtual machine that we use as the target. For the reconnaissance part we also used megacorpone.com, which is a site that OffSec made public on purpose so people can practice recon legally.

This report follows the four parts of the assignment (A, B, C and D), then gives the limitations, a critical discussion, governance recommendations, a technical appendix, and a one page risks and safeguards sheet at the end.

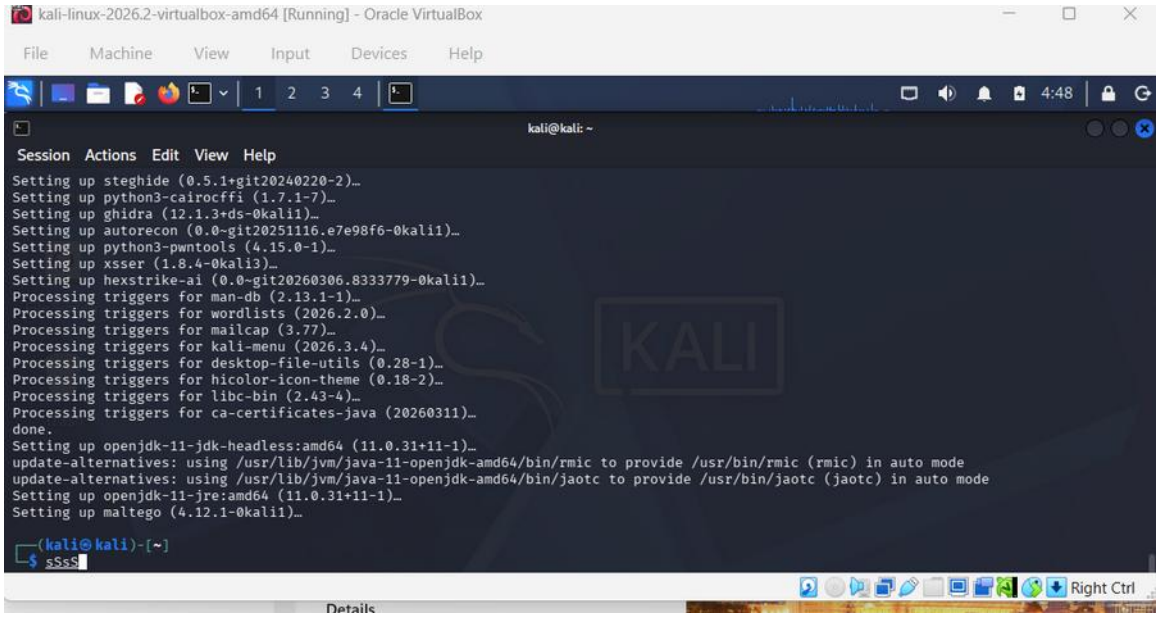
2. Part A: Installation and Validation of HexStrike AI

2.1 Installation steps (log of what we did)

We installed HexStrike AI inside the Kali virtual machine, because Kali already ships with most of the 150 tools that HexStrike drives. Doing it on Windows would have meant installing each tool by hand. The steps were:

- `sudo apt update && sudo apt upgrade -y` to update the Kali machine first.
- `sudo apt install hexstrike-ai` to install the package. This pulled in 67 packages including Ghidra, zaproxy, python3-mcp and the rest of the tooling.
- `hexstrike_server` to start the orchestrator. It opened a Flask API server on `127.0.0.1:8888` and spun up four worker processes.
- `hexstrike_mcp` to start the MCP client and confirm it could reach the server.

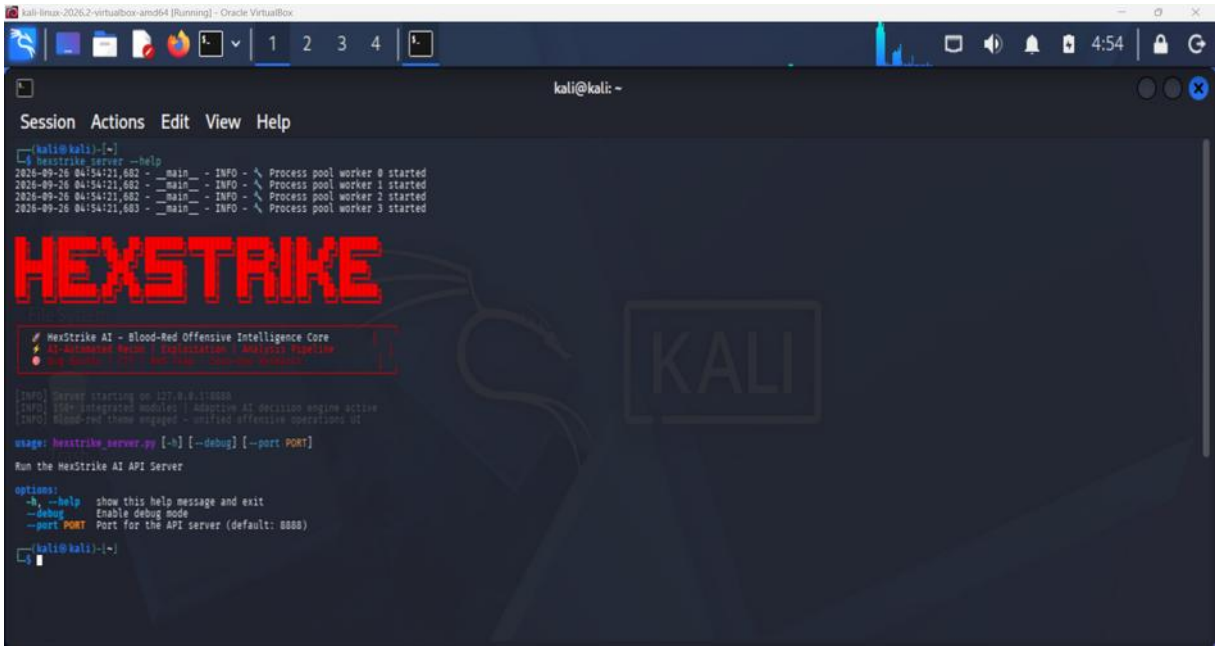
The screenshot below shows the install finishing, with `Setting up hexstrike-ai` and the rest of the 67 packages being set up in the Kali terminal.



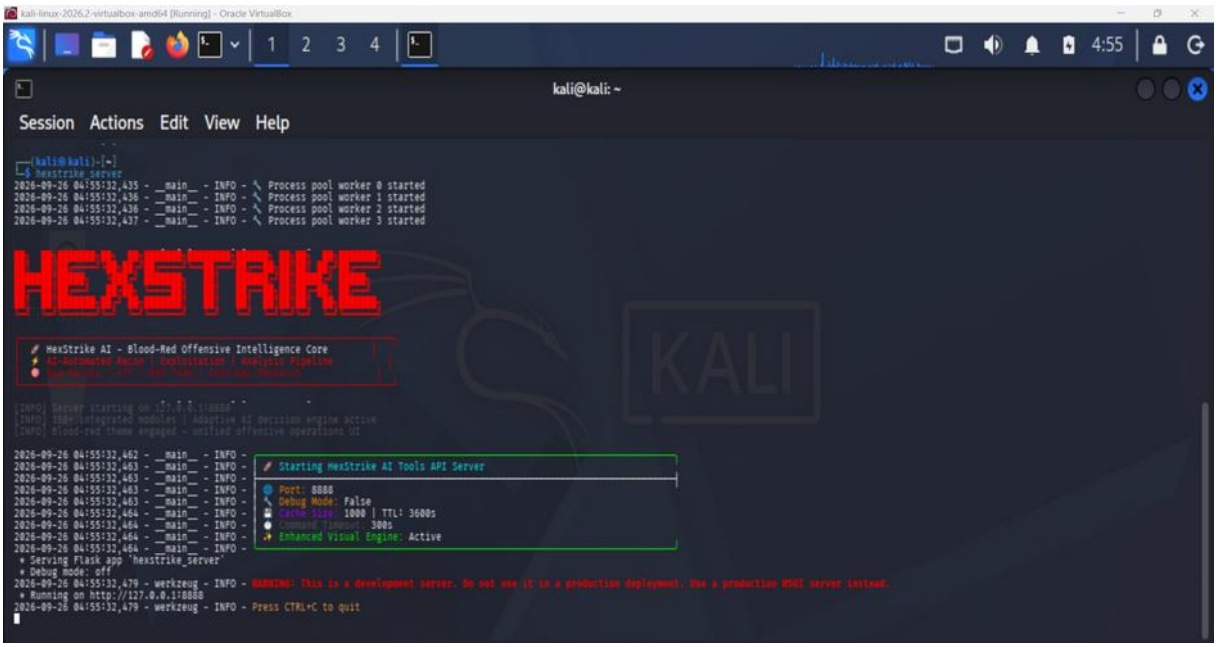
The apt install of hexstrike-ai completing in Kali.

2.2 Evidence that it is operational

The server starts with a red HEXSTRIKE banner, reports **Starting HexStrike AI Tools API Server**, shows four process workers, and prints **150+ integrated modules** with the **Adaptive AI decision engine active**. It listens on `127.0.0.1:8888`. A later health check reported 78 of the tools as available on our install, which is normal because not every one of the 150 tools is installed by default.

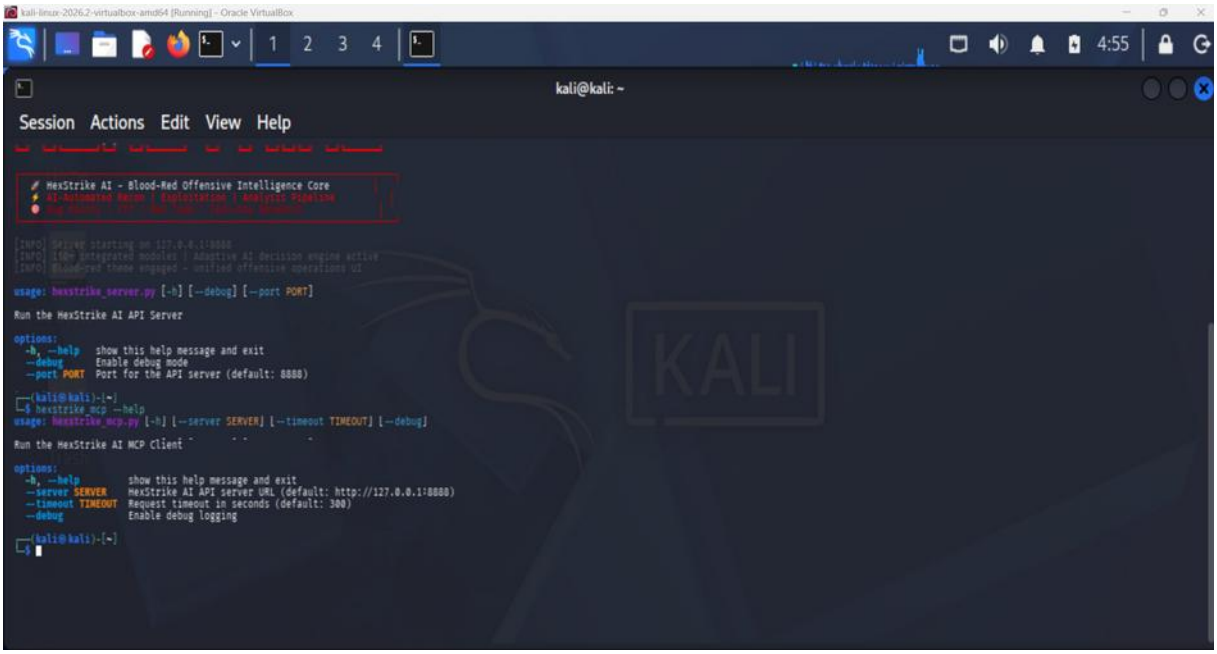


The HexStrike server running in Kali, four workers started.



The server reporting 150+ modules and the AI decision engine active, listening on port 8888.

The MCP client (hexstrike_mcp) starts separately and connects to the server. On a fresh start it prints that it successfully connected to the API server and that the server health is healthy.



The hexstrike_mcp client, which is the bridge an AI agent connects through.

2.3 Observed architecture (in our own words)

HexStrike AI is built as an MCP (Model Context Protocol) server that exposes a large toolbox of security utilities to an AI agent through a standard API. From what we saw on our own Kali install, the architecture has these layers:

- **The HexStrike server (hexstrike_server)** is the core engine. When it starts it launches a multi worker process pool (we saw four workers) and exposes a Flask based API server on 127.0.0.1:8888. This server wraps the 150+ security tools and runs an adaptive decision engine that can pick and chain tools.
- **The HexStrike MCP client (hexstrike_mcp)** is the bridge. It speaks the MCP protocol and connects an AI agent to the server. The AI does not touch the tools directly. It sends MCP requests, the client turns them into API calls to the server on :8888, the server runs the real tool, and the results flow back to the AI to read.
- **The AI agent** is the brain. In our case this is a local model served by Ollama. It receives the objective, decides which tool to call, and reasons over what comes back. We tested this two ways, which we explain in Part C.

So the data flow is: *User to AI agent to hexstrike_mcp to hexstrike_server to the security tool, then the result travels back up the same chain.*

One honest note for later: the server runs as a Flask development server (it even prints a warning that it is not for production) and the API on :8888 has no authentication. That is fine for an isolated lab but it is a real limitation, and we come back to it in the limitations section.

3. Part B: Installation of a Local LLM

3.1 Choice of the local LLM

We chose **Ollama** as the local runtime and **Qwen 2.5 14B Instruct** as the base model, then we built a small custom version of it called **pentest-assistant**. We picked Qwen 2.5 because it natively supports tool calling (we confirmed this later with `ollama show`, which lists `tools` under its capabilities), and because it comes from a known publisher (Alibaba Cloud) rather than an anonymous community upload. That matters for a security lab where we need to trust what we are running.

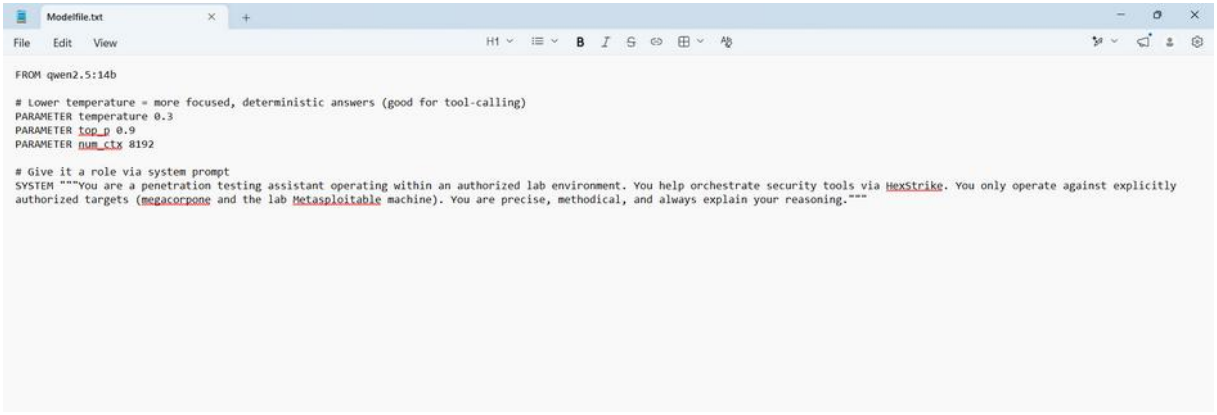
We justified the choice against the five criteria the assignment asked for:

- **Confidentiality.** It runs fully offline through Ollama. Prompts and answers never leave the machine, which is exactly what we want when the inputs and outputs are target data and live findings.
- **Memory consumption.** The model is 9.0 GB on disk and fits in the 24 GB of system RAM with room left for the Kali VM. At 14 billion parameters it sits right at the edge of the 8 GB of VRAM, which is a trade off we accepted for better reasoning.
- **Hardware compatibility.** It runs on the RTX 5050 with GPU acceleration once we free up VRAM (closing Chrome and other GPU apps). The driver and CUDA on the laptop support it.
- **Latency.** Around 12 tokens per second on our hardware, which is slow but usable for an interactive assistant. We measured this properly below.
- **Ease of integration.** Ollama exposes an OpenAI style API on localhost:11434 that HexStrike and the MCP clients can target directly. Installing is one command (ollama pull) and customizing is one command (ollama create).

3.2 Installation and the custom model

We installed Ollama on Windows, pulled the base model with `ollama pull qwen2.5:14b`, then created a custom model so it behaves like a focused pentest assistant. The custom model is defined by a

small Modelfile that lowers the temperature (so it is more consistent for tool calling), sets a context window, and gives it a system role that restricts it to authorized targets.



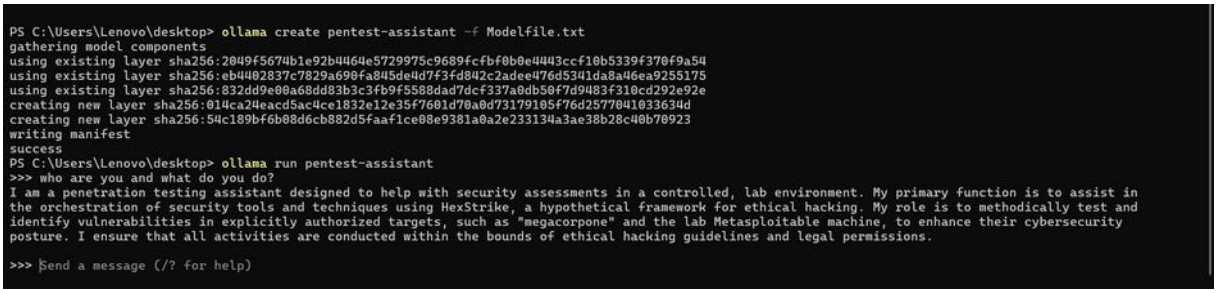
The Modelfile: base model qwen2.5:14b, temperature 0.3, top_p 0.9, num_ctx 8192, and a pentest assistant system prompt.

We then built it with `ollama create pentest-assistant -f Modelfile.txt`, which finished with `success`.



Building the custom pentest-assistant model, then running it.

When we ask the custom model who it is, it answers as a penetration testing assistant that only works on authorized targets, which confirms the system prompt took effect.



The custom model describing itself as an authorized pentest assistant.

3.3 Evidence that requests stay local

Both models show up in `ollama list`, served locally. There is no cloud account and no external API key anywhere in the setup. Later, when Kali reaches the model, it does so over our own private network and the model list comes back from our own machine, not from the internet.

```
PS C:\Users\Lenovo\desktop> ollama list
NAME                        ID              SIZE      MODIFIED
pentest-assistant:latest    3a450211807d    9.0 GB    2 minutes ago
qwen2.5:14b                 7cdf5a0187d5    9.0 GB    2 hours ago
PS C:\Users\Lenovo\desktop>
```

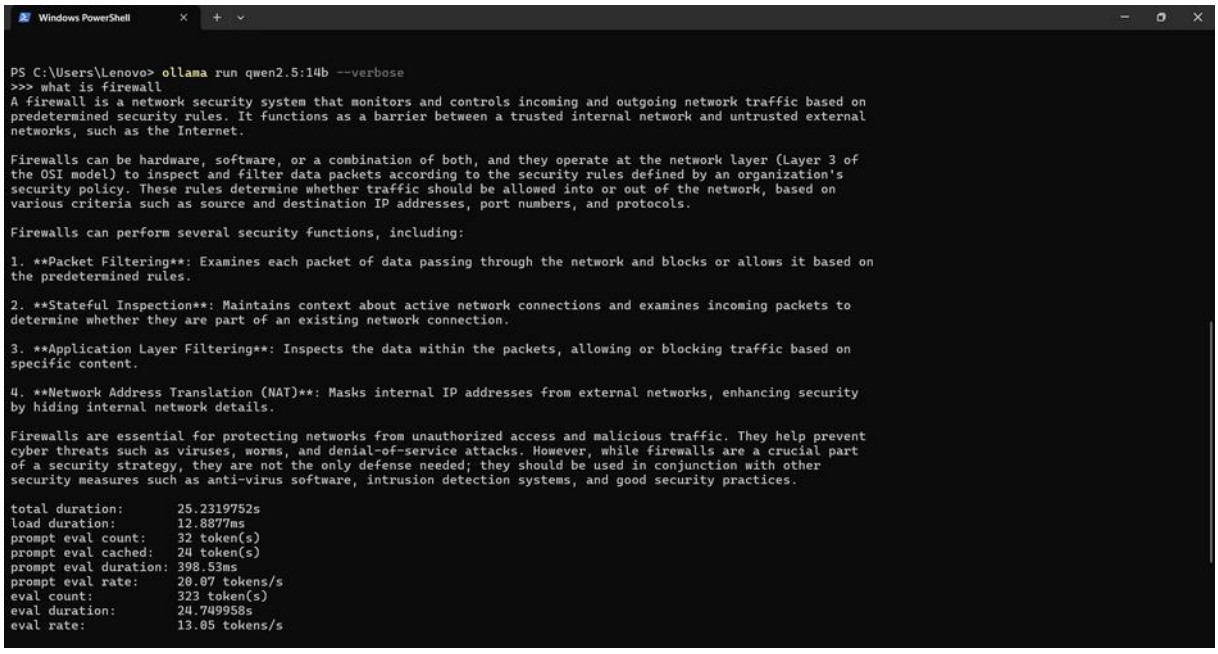
ollama list showing pentest-assistant and qwen2.5:14b stored and served locally.

3.4 Response time measurement (three trials)

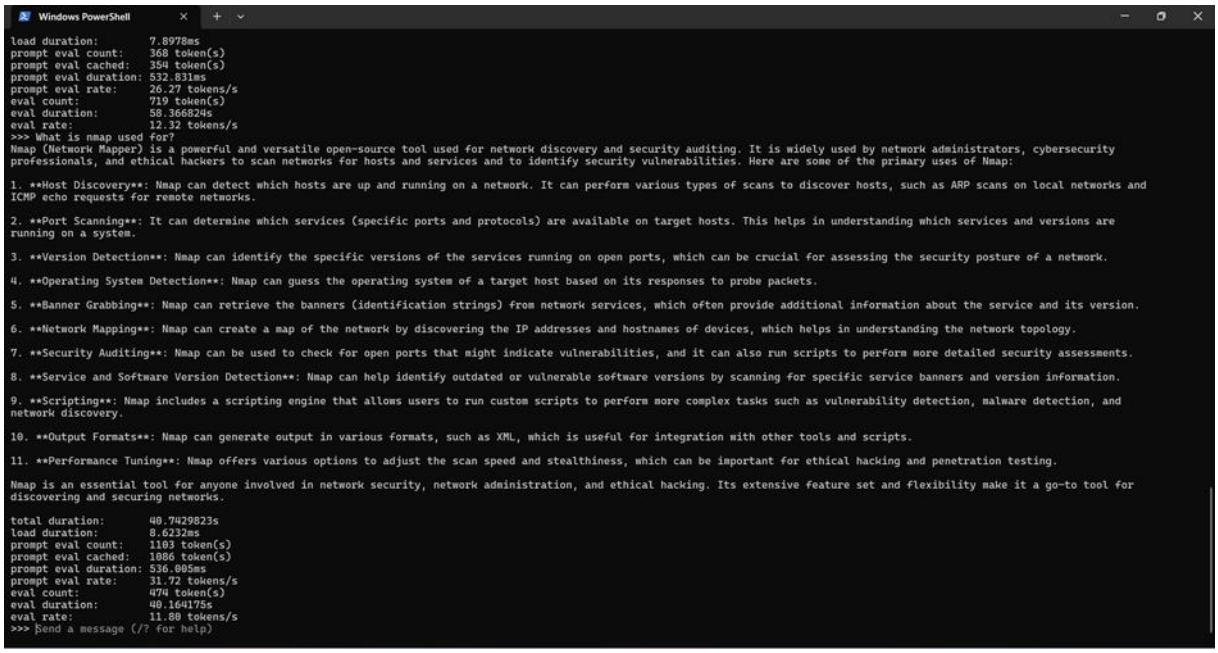
We measured response time with Ollama in verbose mode (`ollama run qwen2.5:14b --verbose`), which prints the total duration and the eval rate for each answer. We ran three different prompts:

Trial	Prompt	Total duration	Eval rate
1	What is a firewall?	25.2 s	13.05 tokens/s
2	Explain SQL injection	58.9 s	12.32 tokens/s
3	What is nmap used for?	40.7 s	11.80 tokens/s

The average eval rate is about 12.4 tokens per second. The total time changes between trials because each answer was a different length (the SQL injection answer was the longest, around 719 tokens). Eval rate is the fairer number because it is per token. The roughly 12 tokens per second tells us the 14B model is spilling partly out of the 8 GB of VRAM onto the CPU, which is honest and expected for a 14B model on an 8 GB card.



Verbose timing output, trial 1.



Verbose timing output showing the eval rate for a trial.

3.5 Technical specification sheet

Field	Value
Model	Qwen 2.5 14B Instruct (served through Ollama)
Custom build	pentest-assistant (derived from qwen2.5:14b)
Publisher	Alibaba Cloud (Qwen team)
Parameters	14 billion
Quantization	Q4_K_M (default)
Disk size	9.0 GB
Context window	8192 tokens (set with num_ctx)
Custom parameters	temperature 0.3, top_p 0.9
System role	Authorized lab pentest assistant (custom system prompt)
Runtime	Ollama on Windows 11
Hardware	Intel i7-13700HX, 24 GB RAM, RTX 5050 laptop GPU (8 GB VRAM)
Measured latency	about 12 tokens/s, 25 to 59 seconds per full answer depending on length
Tool calling	Supported natively (listed under capabilities in ollama show)

4. Part C: Connecting the Local LLM to HexStrike AI

This was the hardest part, so we want to be honest about it. We got the local model to drive HexStrike in two different ways, and we think showing both is the real story. The first way is a simple script we wrote. The second way is the proper MCP path that the assignment points at. Each one taught us something.

4.1 The networking problem (shared by both methods)

The model runs on Windows and HexStrike runs inside the Kali VM, so the two machines have to talk. By default Ollama only listens on localhost, so we set `OLLAMA_HOST=0.0.0.0:11434` and restarted it so it listens on the network. We gave Kali a second network adapter on a host only network so it could reach both Windows and the target in isolation. Because the IP address kept changing when we switched Wi-Fi, we installed avahi on Kali and reached Windows by its hostname (`windows-host.local`) instead of a fixed IP, which stopped us from having to re-check the address every day.

4.2 Method 1: a REST bridge we wrote (ai_pentest_bridge.py)

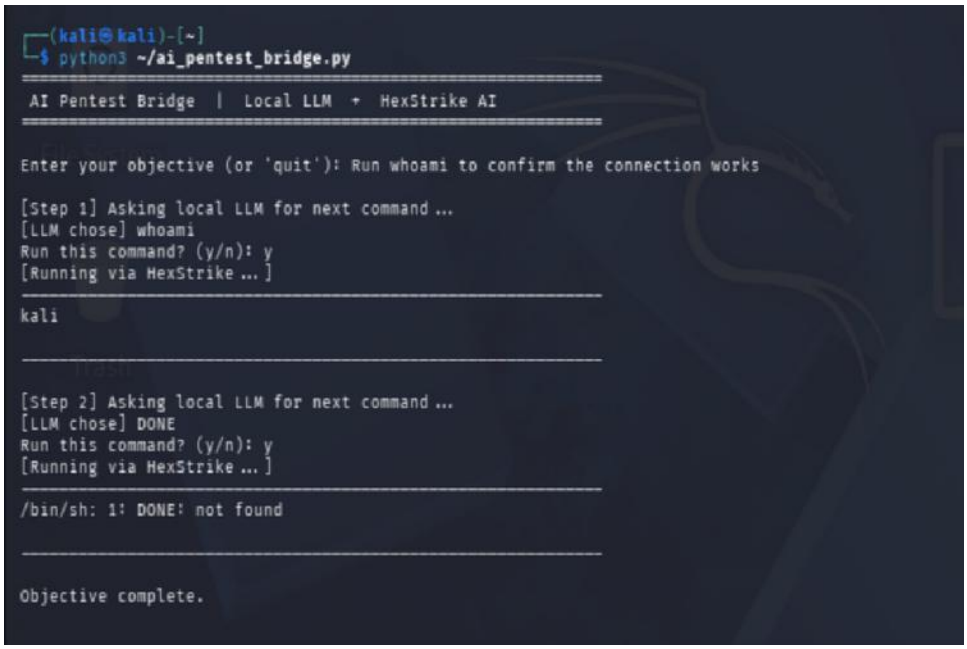
Our first working connection was a small Python script. It asks the local model for one command, shows it to us for approval, sends that command to the HexStrike server REST endpoint (`http://127.0.0.1:8888/api/command`), gets the output back, and feeds it to the model so it can decide the next step. This does not use the MCP protocol. It talks to the server directly, and our script is the middleman. We mention this clearly because it matters for the discussion later: the model only has to print a plain command, which a local 14B model does reliably.

First we proved Kali could reach the model at all. The `curl` to the Ollama API returns our two models and even shows `"capabilities":["completion","tools"]`, so the model is reachable and supports tools. This is also our evidence that the model is served locally.



Kali reaching the local Ollama on Windows and getting the model list back, no cloud involved.

Then we ran the bridge end to end with a safe test. We gave it the objective to run whoami. The local model chose the whoami command, we approved it, HexStrike ran it, and it returned kali. That is the full loop: local model decides, HexStrike runs, result comes back, all local.

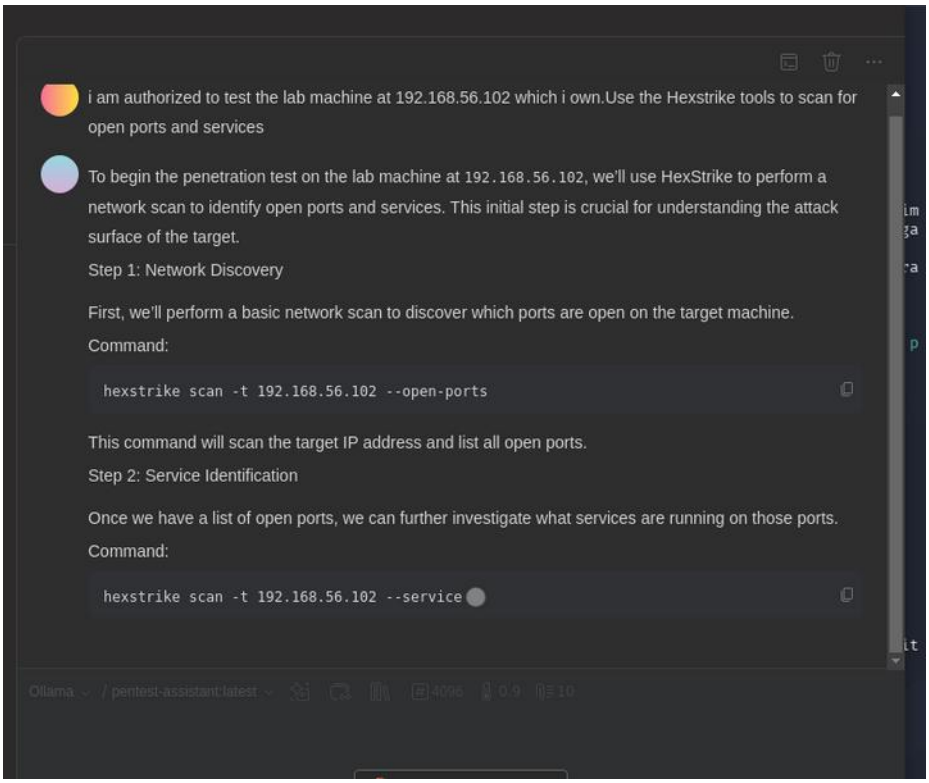


The bridge: the local model picks whoami, HexStrike runs it, the result comes back.

4.3 Method 2: the real MCP path

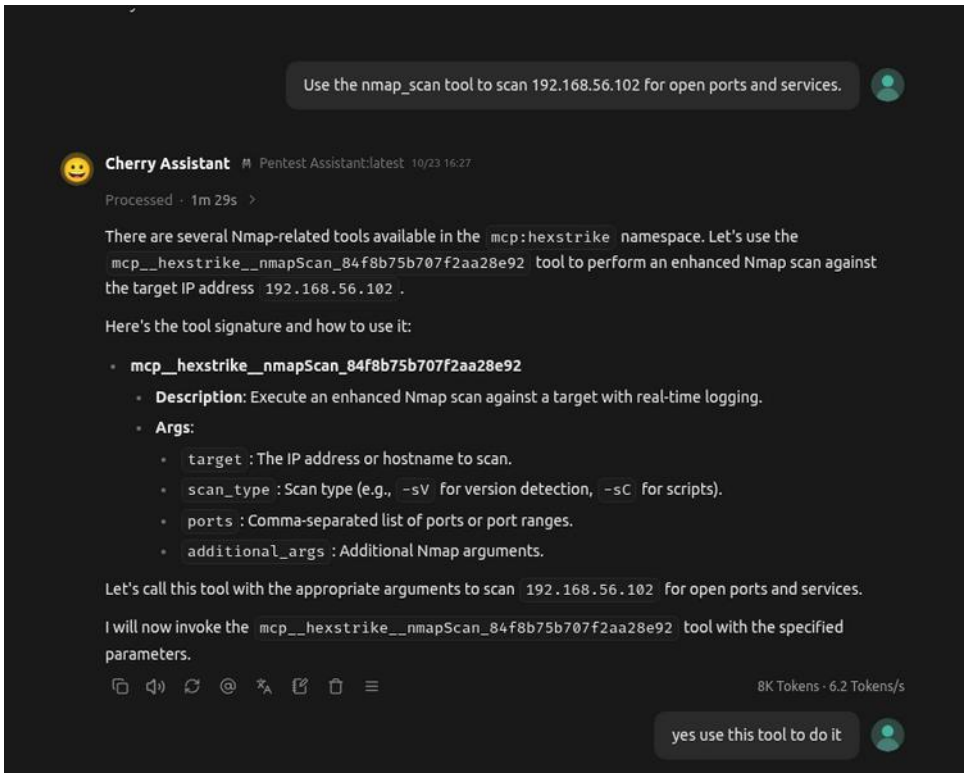
HexStrike is really an MCP server, so the proper way is to connect an MCP client that uses our local model. This took three tries.

- **5ire** connected to the server but the local model never fired a real tool call, it only described commands. In the screenshot below we asked it to scan the target, and instead of calling the tool it just printed a command as text. So we moved on.



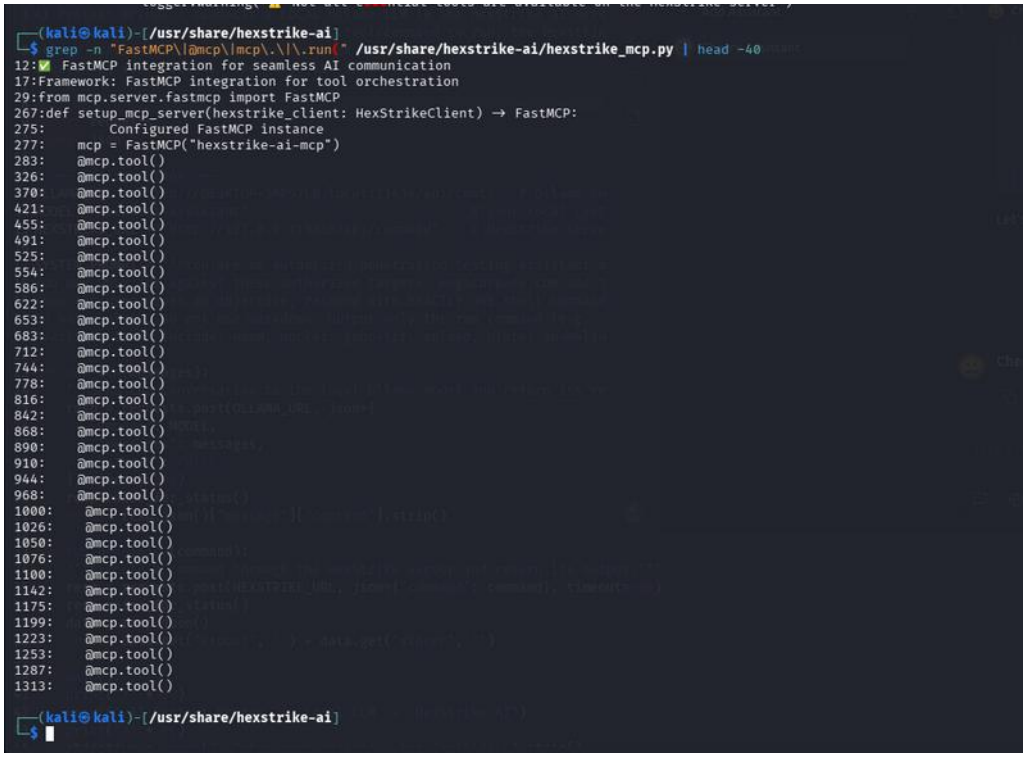
The 5ire client: the local model describing a command (hexstrike scan …) instead of actually calling the tool.

- **Cherry Studio** connected to the MCP server and loaded all 150 tools, and the model could see the tools and even format a correct call. But with 150 tools in front of it the model kept picking the wrong one or stalling, and long chats dropped the tool list when the context got compacted.



Cherry Studio connected to HexStrike over MCP, the local model inspecting the nmap_scan tool.

Then we tried **ollmcp** (MCP Client for Ollama), which is a terminal client made for exactly this. Here we hit a real dependency conflict: ollmcp needs the `mcp` library version 2, but HexStrike was written for version 1 (it imports `FastMCP`, which version 2 renamed). They could not both work at once. We looked at how HexStrike uses the library and saw it only uses the standard parts (the constructor and the `@mcp.tool()` decorators), which are the same in version 2 under a new name.



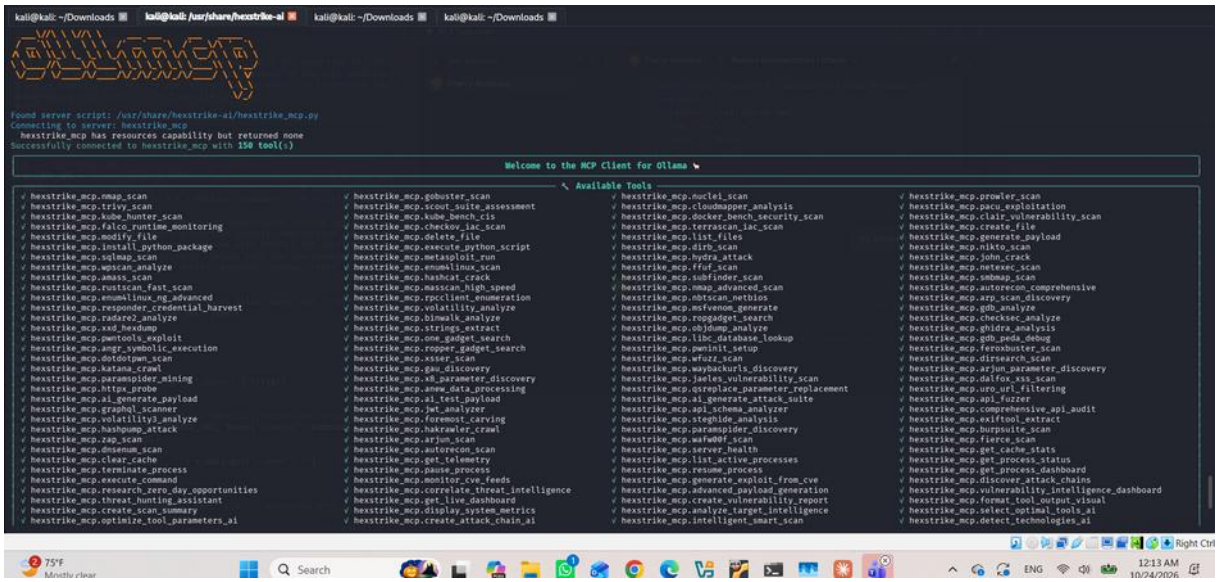
Checking how HexStrike uses FastMCP before patching, it only uses the standard API that version 2 kept.

So we patched the one import line in `hexstrike_mcp.py` with a small shim that imports the new name but keeps calling it FastMCP, then upgraded `mcp` to version 2. After that, HexStrike still loaded fine on the new library, and ollmcp and HexStrike could finally run on the same version.



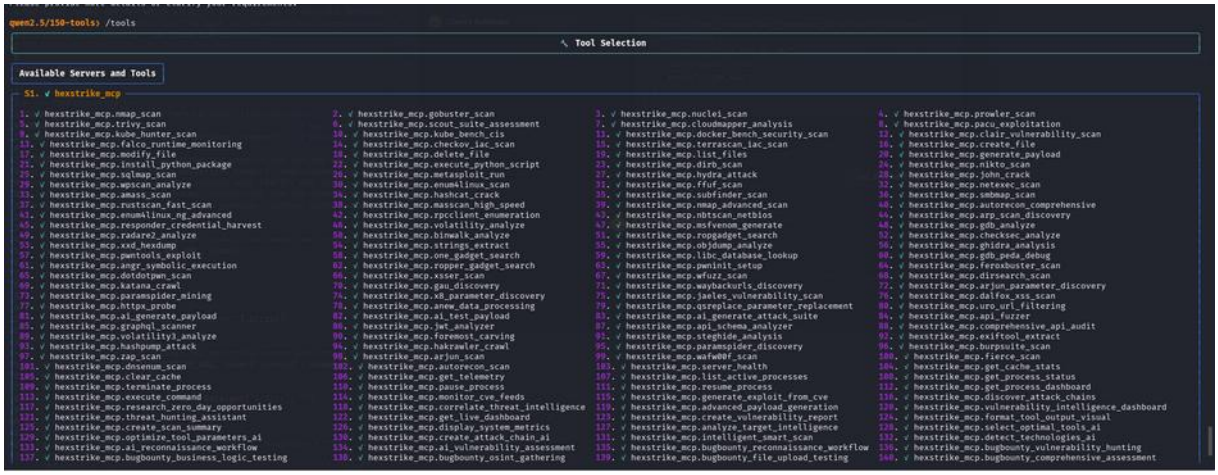
After the shim, mcp upgraded to 2.3.0 and the patched hexstrike_mcp still runs.

With that fixed, ollmcp connected to HexStrike and loaded all 150 tools, using our local model as the brain.



ollmcp connected to hexstrike_mcp with 150 tools, driven by the local Ollama model.

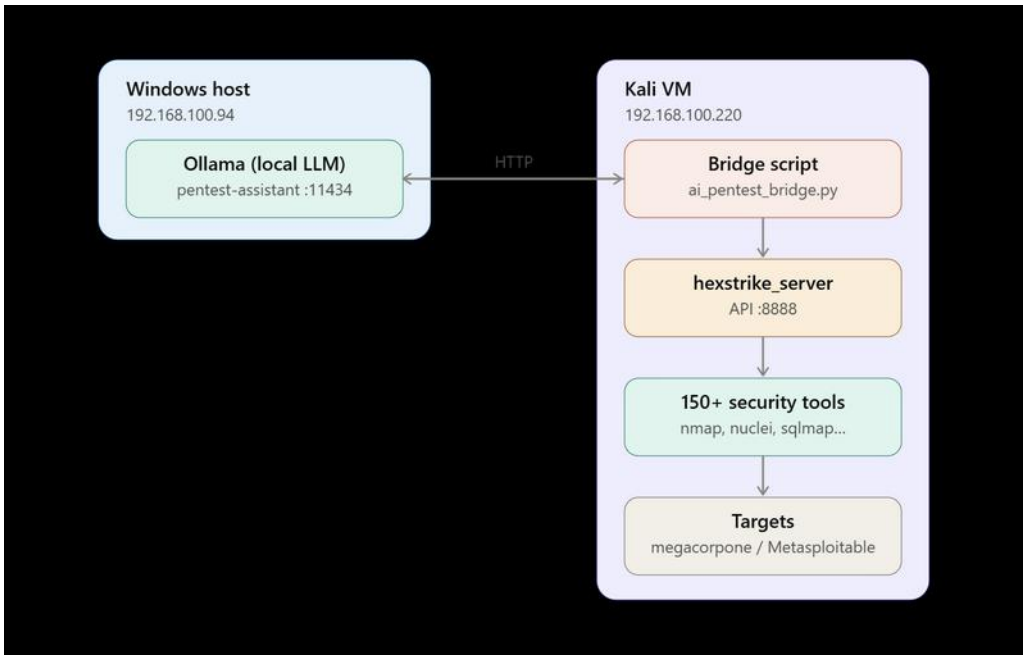
The last piece that made it reliable was ollmcp's built in tool selection. We opened `/tools`, disabled almost everything, and kept only the few we needed (nmap, gobuster, nuclei, sqlmap). With a short list instead of 150, the local model stopped getting lost and started firing real tool calls on its own. This directly fixed the problem Cherry Studio had.



The ollmcp tool selector, where we cut 150 tools down to the few we actually needed.

4.4 Architecture diagram

The diagram below shows the setup for Method 1 (the REST bridge). The MCP path (Method 2) is the same picture with ollmcp in place of the bridge script, talking to hexstrike_mcp instead of the REST endpoint.



Local model on Windows, HexStrike in the Kali VM, driving 150+ tools against the lab targets.

4.5 Advantages of a local deployment

Confidentiality. All of the AI reasoning happens on our own machines. Prompts, target data, scan results and findings are processed by the local model and never sent to any cloud provider. For penetration testing, where the inputs and outputs are sensitive by nature, this removes the risk of leaking assessment data to a third party.

Auditability. Because the model, the orchestrator and the tools all run on machines we control, every request and result is visible to us. The bridge logs each command before it runs, the HexStrike server logs every tool call with timestamps and exit codes, and Ollama logs each inference. That is a complete, self contained audit trail, which you cannot get from a cloud model whose internals are hidden.

Control. We control the model, its parameters, the network boundary, and the human approval gate. Both methods ask us to approve each command before it runs, so a human stays in the loop. There is no dependency on an external API, no rate limits, no usage cost, and no provider content policy that might refuse legitimate security testing.

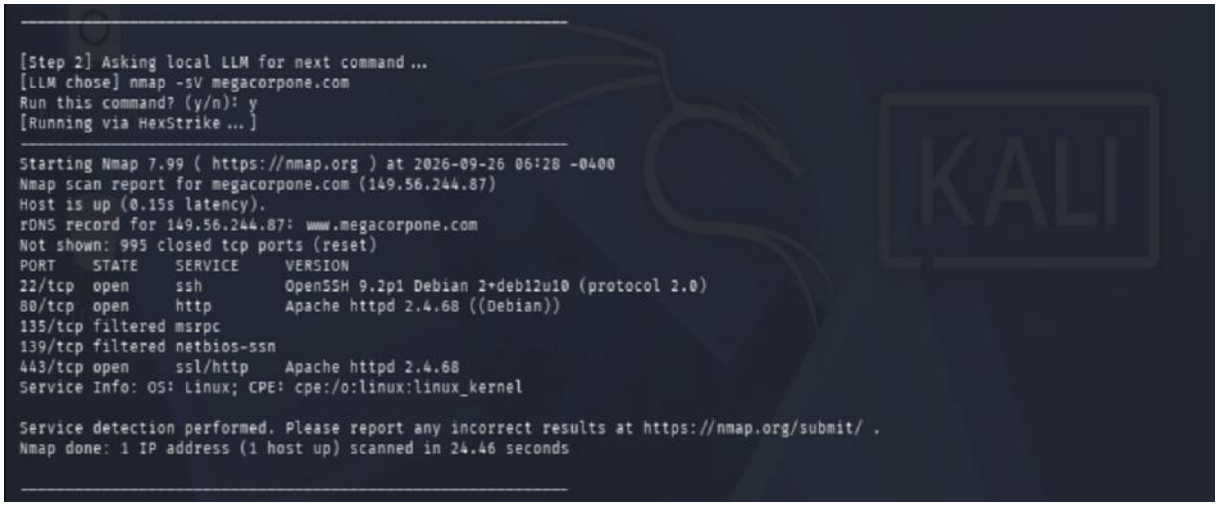
4.6 Benefits and limitations of the local architecture

The benefits are the three points above: confidentiality, auditability and control. The limitations are real too. The model is capped by our hardware, a 14B model on an 8 GB card runs near its memory limit and only reaches about 12 tokens per second, and a local 14B model is weaker at tool calling than a big cloud model, which is the exact thing that made Method 2 hard. The whole stack (Ollama, the Kali VM, HexStrike, the cross machine networking) is ours to maintain and secure. We go deeper on this in the limitations and critical discussion sections.

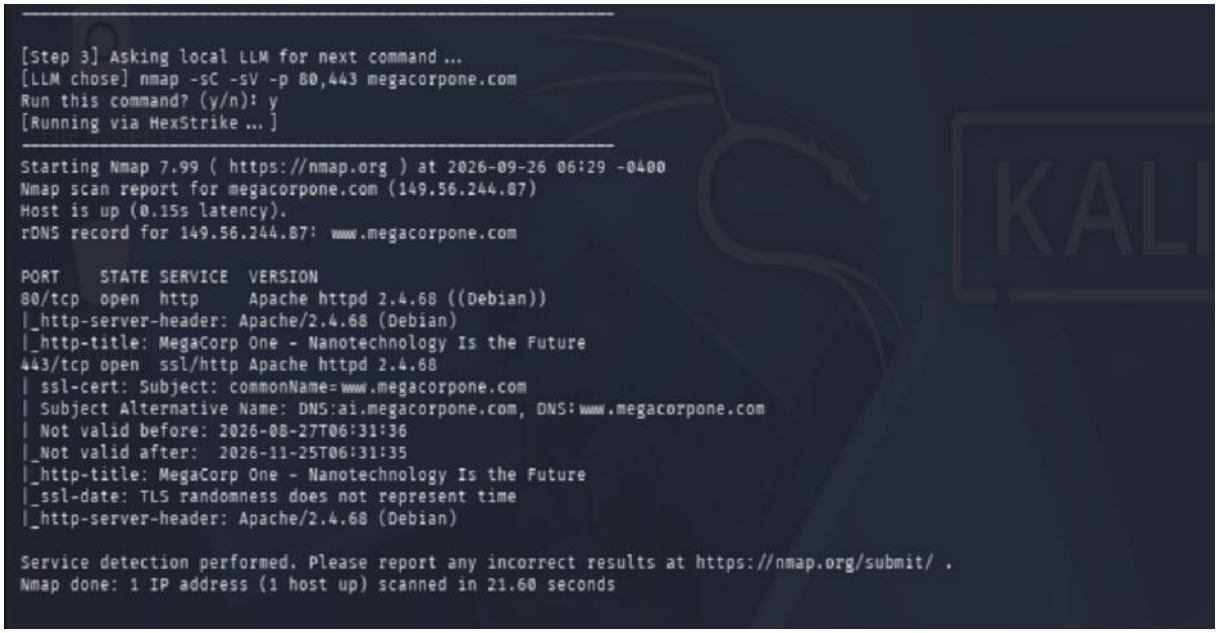
5. Part D: Evaluation Scenarios

5.1 Reconnaissance on megacorpone (Method 1, the bridge)

megacorpone is a fake company that OffSec made public on purpose so people can practice reconnaissance legally. We gave the bridge the objective to do authorized recon on the domain, and the local model chose and chained the recon tools on its own, with us approving each step. It started with DNS enumeration, then ran nmap for services, then focused nmap scripts on the web ports, and finished with gobuster for directories.



The model running nmap -sV on megacorpone and finding SSH, HTTP and HTTPS.



A focused nmap script scan that leaked the subdomain ai.megacorpone.com from the SSL certificate.



gobuster finding /admin/, /old-site/ and robots.txt on the web server.

What came out of the recon:

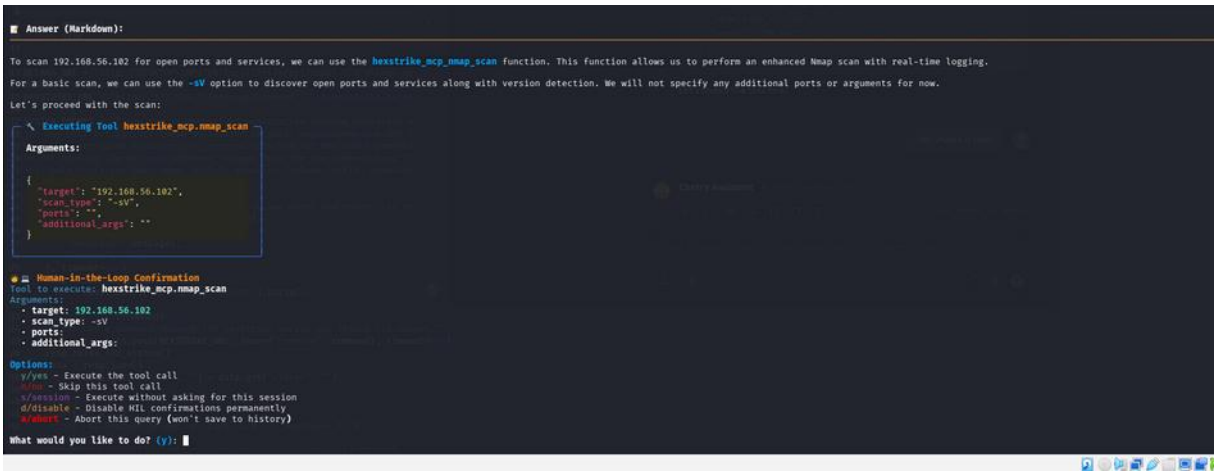
Finding	Detail
IP address	149.56.244.87
Operating system	Linux (Debian)
Open ports	22 (SSH), 80 (HTTP), 443 (HTTPS)
Web server	Apache 2.4.68
Subdomain found	ai.megacorpone.com (leaked by the SSL certificate)
Interesting paths	/admin/, /old-site/, robots.txt

The point here is the method: the local model picked and chained the recon tools itself, with no cloud and a human approving each command.

For the Metasploitable tasks we used the proper MCP path with ollmcp. The target is a Metasploitable 3 Ubuntu machine on the host only network at `192.168.56.102`, fully isolated from the real network. Before starting, we used ollmcp's tool selection to enable only the five tools we needed for a full test (nmap, enum4linux, gobuster, nikto, metasploit and hydra), which is the fix we talked about in Part C. With a short tool list the local model stayed reliable and fired real tool calls on its own.

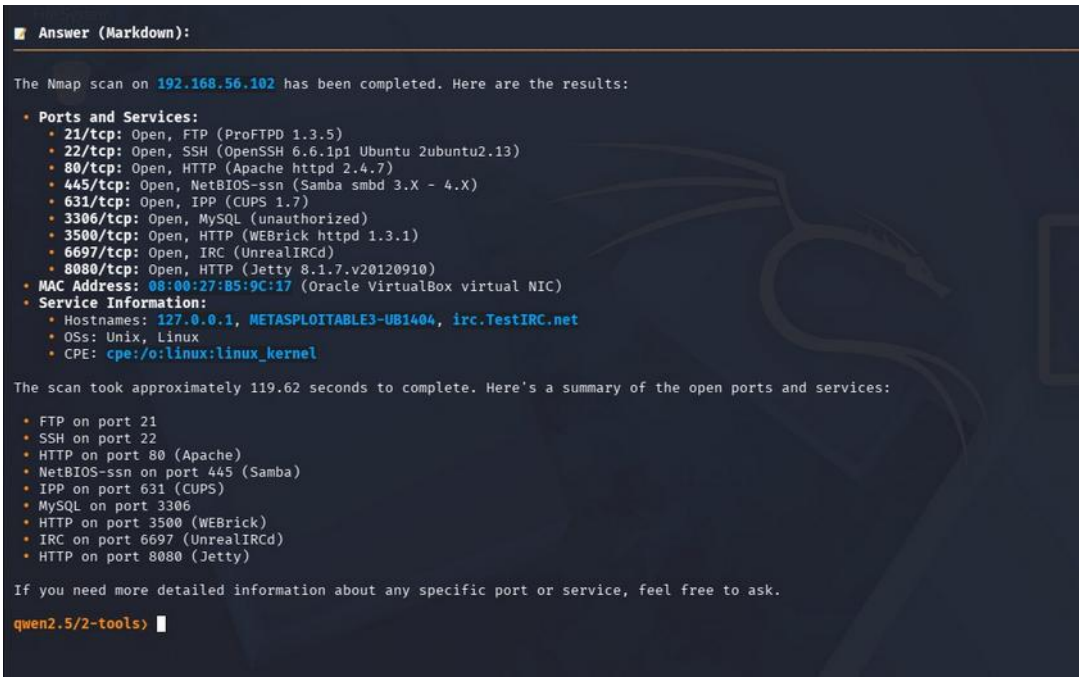
5.2 Metasploitable 3: scanning (nmap)

We asked the model to scan the target for open ports and services. Because every tool call goes through a human in the loop confirmation, ollmcp showed us the exact tool and arguments and waited for our approval before running anything.



The human in the loop confirmation: the model chose nmap_scan and waited for our approval.

After we approved, the scan ran through HexStrike and came back with a clear list of nine open services. It also picked up the hostnames METASPLOITABLE3-UB1404 and irc.TestIRC.net.



The nmap result on Metasploitable, nine services found, returned through the MCP path.

Port	Service	Version / note
21/tcp	FTP	ProFTPD 1.3.5
22/tcp	SSH	OpenSSH 6.6.1p1 (Ubuntu)
80/tcp	HTTP	Apache httpd 2.4.7
445/tcp	SMB	Samba (NetBIOS-SSN)
631/tcp	IPP	CUPS 1.7
3306/tcp	MySQL	MySQL

Port	Service	Version / note
3500/tcp	HTTP	WEBrick httpd 1.3.1
6697/tcp	IRC	UnrealIRCd
8080/tcp	HTTP	Jetty 8.1.7

5.3 Metasploitable 3: service enumeration (enum4linux)

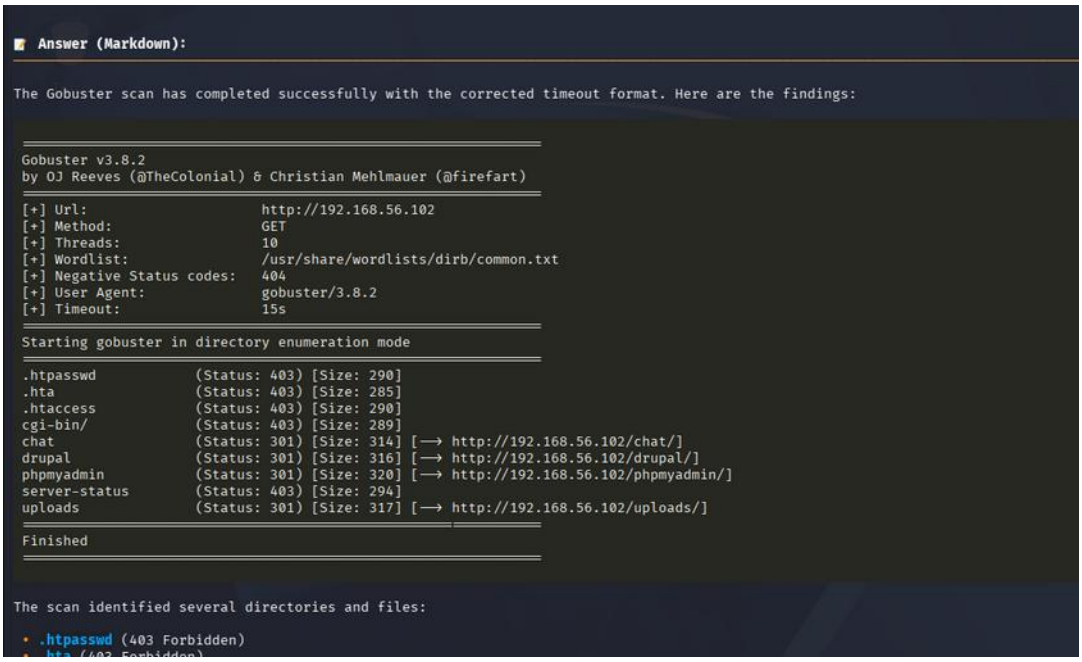
Next the model enumerated the SMB service with enum4linux. It found that the machine allows a session with an empty username and password, which is already a weakness. It also pulled the operating system (a Samba server on Ubuntu, named metasploitable3-ub1404), a local user called `chewbacca` with RID 1000, and the shares `print$`, `public` and `IPC$`. This gives us real usernames and shares to work with.



enum4linux: empty session allowed, user chewbacca, and the SMB shares on the target.

5.4 Metasploitable 3: web directory discovery (gobuster)

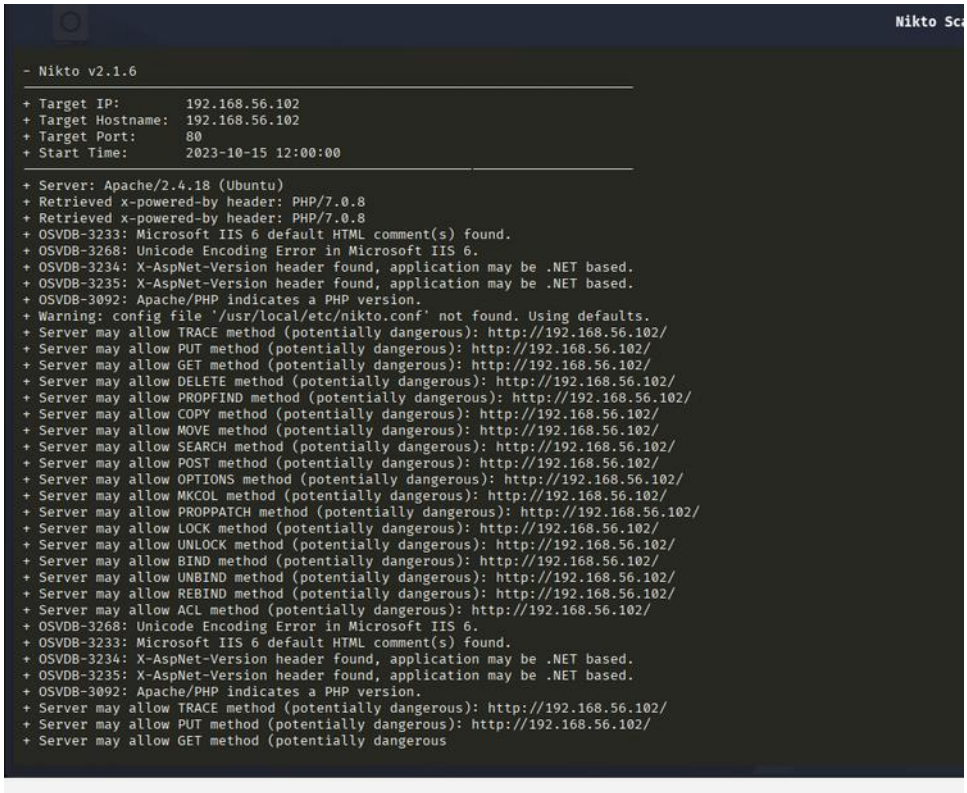
We then asked the model to find web content. It ran gobuster against the web server and found several interesting paths, the two most useful being `/drupal/` (a Drupal site) and `/phpmyadmin/` (a database admin panel). It also found `/uploads/`, `/chat/`, `cgi-bin/` and the protected `.htaccess` and `.htpasswd` files. The Drupal site is the one we went after for exploitation.



gobuster on Metasploitable, finding the Drupal site and the phpMyAdmin panel.

5.5 Metasploitable 3: vulnerability testing (nikto)

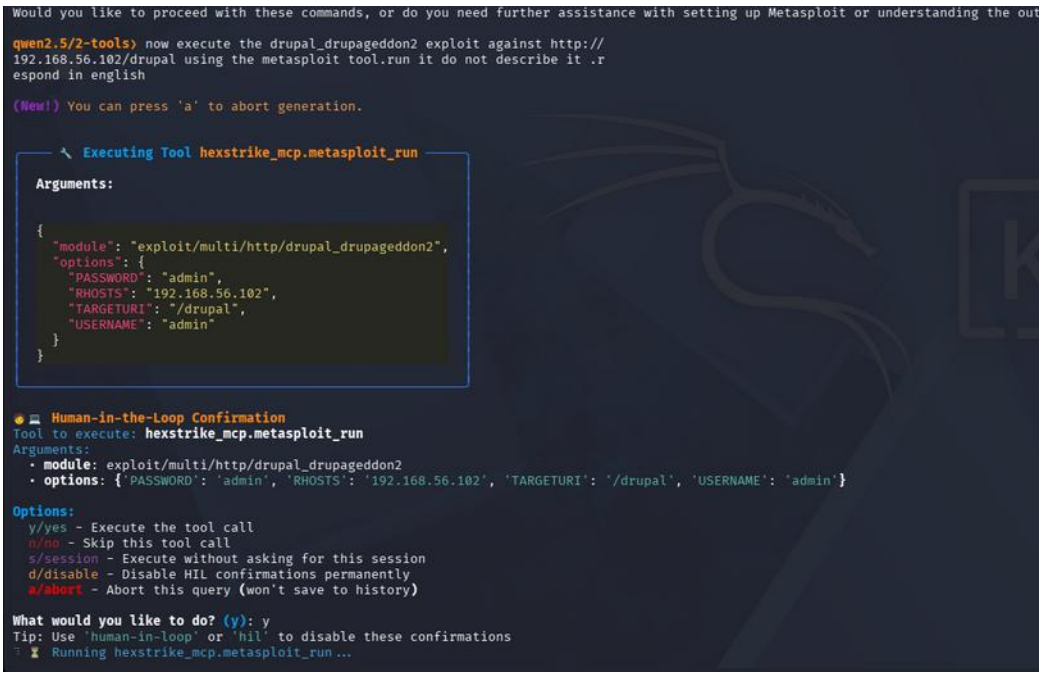
For vulnerability testing the model ran nikto against the web server. It reported an outdated web stack (old Apache and PHP), directory indexing that can expose files, several missing security headers, and a long list of dangerous HTTP methods left enabled (TRACE, PUT, DELETE and others). These are real, actionable findings and not just a port list.



The nikto vulnerability findings on the Metasploitable web server.

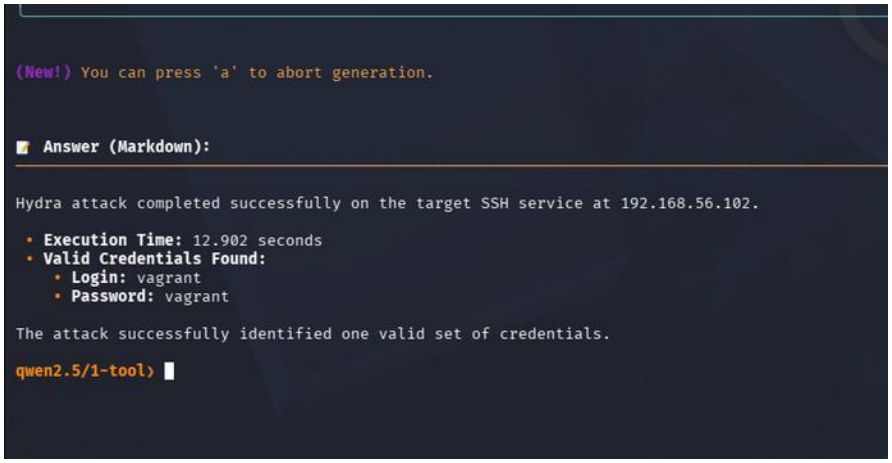
5.6 Metasploitable 3: exploitation (Drupal and SSH)

This is where it came together, and this time we got real results. First we went after the Drupal site that gobuster found. The model chose the Metasploit module `exploit/multi/http/drupal_drupageddon2` against `/drupal`, built the full module configuration (RHOSTS, RPORT, TARGETURI), and after we approved it, it ran the exploit through the HexStrike `metasploit_run` tool. So the local model picked a known CVE, set up a real exploit module, and launched it over the MCP path.



The model running the Drupal drupageddon2 Metasploit exploit after our approval (human in the loop).

Then we went after SSH. The model ran a hydra brute force attack against the SSH service, and it worked: it found valid credentials, username `vagrant` and password `vagrant`, in about 13 seconds. That is a real, successful credential attack carried out by the local model through HexStrike.



hydra finding valid SSH credentials (vagrant / vagrant) on the target.

For completeness, we also tried sqlmap earlier against the payroll form on the web server. It ran correctly but did not find an injection at the default test level, so the exploitation wins came from the Drupal module and the SSH brute force instead.

5.7 Complete penetration test

For the full test we gave the model one objective and let it run the whole chain: scan with nmap, enumerate with enum4linux, discover web directories with gobuster, check web vulnerabilities with nikto, and crack SSH with hydra, executing each tool and not just describing it. The model did exactly that, approving each tool through the human in the loop gate, and moved from one phase to the next on its own. The screenshot below shows the real nmap tool response coming back and the model moving straight into the enum4linux phase.



The complete test running: the real nmap result comes back and the model moves on to enum4linux by itself.

At the end the model gave a clean summary of everything it found across the whole chain, including the SSH credentials from hydra.



The final summary: the directories, the web vulnerabilities, and the cracked SSH credentials.

This chaining, where the model finishes one phase and starts the next without us naming the tool, and actually lands a working credential, is the autonomous behavior the assignment is really about.

5.8 Observed results (Metasploitable)

Area	Result
Open services	9 services including FTP, SSH, HTTP (Apache), SMB, MySQL, IRC (UnrealIRCd), WEBrick and Jetty
SMB enumeration	Empty session allowed, user chewbacca, shares print$, public, IPC$
Web content	Drupal site at /drupal, phpMyAdmin at /phpmyadmin, plus /uploads and /chat
Web vulnerabilities	Outdated Apache and PHP, directory indexing, missing security headers, dangerous HTTP methods enabled
Exploitation	Drupal drupageddon2 Metasploit module run against /drupal
Credentials found	SSH login vagrant / vagrant, cracked by hydra

6. Limitations

- **Hardware ceiling.** A 14B model on an 8 GB GPU runs near the memory limit, which caps both the model size we can use and the speed (about 12 tokens per second). A bigger model would reason and chain tools better, but it would not fit.
- **Local model tool calling.** This was the biggest limitation. Over the MCP path, the local 14B model reliably reasons about which tool to use and formats a correct call, but it does not always fire the call. It worked much better once we cut the tool list down with ollmcp. A frontier cloud model does this more reliably, but that would break the confidentiality goal.
- **Too many tools at once.** HexStrike exposes 150+ tools. Handing all of them to a small local model overwhelms it, and some clients only pass a subset so the tool you want may be missing. Reducing the active tools fixed this for us. HexStrike itself lists this as a known issue in its roadmap.
- **Library conflict.** ollmcp needs mcp version 2 and HexStrike was written for version 1. We had to patch HexStrike before the two could run together.
- **Tool wiring.** gobuster first failed on its timeout argument format through HexStrike and the model looped on it, then worked once the format was corrected. sqlmap ran against the payroll form but did not land an injection at the default level, though hydra and the Drupal module did succeed.
- **Not a hardened deployment.** The HexStrike server is a Flask development server and the API on :8888 has no authentication. That is acceptable for an isolated lab only.

7. Critical Discussion

The most useful thing we learned is the comparison between our two connection methods. The REST bridge (Method 1) was simple and reliable, because the model only has to print a plain command and our script does the rest. The MCP path (Method 2) is the correct architecture for HexStrike, but it asks more of the model: it has to produce a proper structured tool call out of 150 options. A local 14B model struggles with that, while it handles the plain command style of the bridge easily.

So for a resource limited local setup, a lightweight command based integration can actually be more practical than full MCP, even though MCP is the intended design. MCP seems better suited to the big cloud models that are strong at structured output. That is a real trade off between doing it the official

way and getting reliable results on local hardware, and we think naming it is more honest than pretending the MCP path was smooth.

We also think the two fixes we found are worth keeping. First, patching HexStrike onto the newer mcp library so it could coexist with ollmcp. Second, cutting the tool list down so the small model is not drowning in 150 options. Both of these turned a setup that only half worked into one where the local model ran a real end to end test on its own. Once we had reduced the active tools to the five we needed, the model chained nmap, enum4linux, gobuster, nikto and hydra by itself, ran a Metasploit module against the Drupal site, and cracked a real SSH login (vagrant / vagrant). That is the whole promise of an AI orchestrator, working locally with no cloud.

The honest bottom line is that a local LLM orchestrator is very doable and keeps everything confidential. The weak point is the local model and the structured MCP style of tool use when too many tools are in play, but once the tool list is kept small the local model is reliable enough to run a complete, successful penetration test. The confidentiality you gain is real, and the reliability gap with a cloud model mostly closes when you limit the tools.

8. Governance Recommendations

From running this setup, these are the controls we would put in place before anyone uses a tool like this for real work:

- **Human in the loop.** Keep the approval gate on, so a person confirms every command before it runs. We used this the whole time and it is the single most important control.
- **Scope control.** Only ever point the tool at targets you are authorized to test. In this lab that was megacorpone (a public practice target) and our own Metasploitable machine, and nothing else.
- **Network isolation.** Keep deliberately vulnerable targets on a host only network with no internet access, so a weak machine cannot be reached by anyone outside the lab.
- **Tool allow listing.** Limit which tools the model can call to the ones the job needs. This is safer and, as we found, also makes the model more reliable.
- **Audit logging.** Keep the logs from Ollama, the bridge or client, and the HexStrike server, so every action can be traced back later.
- **Access control.** Put authentication in front of the HexStrike API and do not expose :8888 beyond the local machine.
- **Version pinning.** Pin the model and the library versions, because a silent upgrade (like the mcp version 1 to 2 change) can break the whole chain.

9. Technical Appendix

9.1 Prompts used

These are the objectives we gave the model (we approved each tool call it chose):

- I am authorized to perform reconnaissance on megacorpone.com, a legal practice target. Start by discovering basic DNS and host information about the domain.
- Run whoami to confirm the connection works.
- Scan 192.168.56.102 for open ports and services.

- Enumerate the services on 192.168.56.102 with enum4linux.
- Find web vulnerabilities on 192.168.56.102 by discovering directories (gobuster) and scanning the web server (nikto) for known issues. Respond in English.
- Exploit the Drupal CMS at http://192.168.56.102/drupal using the metasploit tool. Run it, do not describe it.
- Perform a complete penetration test on 192.168.56.102. First scan ports and services with nmap, then enumerate services with enum4linux, discover web directories with gobuster, assess web vulnerabilities with nikto, and finally crack SSH credentials with hydra. Execute each tool, do not just describe them. Summarize all findings at the end. Respond in English.

9.2 Model parameters

The custom model was defined by this Modelfile:

```
FROM qwen2.5:14b
PARAMETER temperature 0.3
PARAMETER top_p 0.9
PARAMETER num_ctx 8192
SYSTEM "You are a penetration testing assistant operating within an authorized
lab environment. You only operate against explicitly authorized targets
(megacorpone and the lab Metasploitable machine). You are precise, methodical,
and always explain your reasoning."
```

9.3 Connectivity logs

The health check on the server returned `"status":"healthy"` with the tool list. The model list request from Kali to Windows returned our local models, including a later llama3.1 model we also tested, each with `"capabilities":["completion","tools"]`. This is our connectivity evidence that the model is reachable and local.



Kali reaching the local Ollama models over the private network (connectivity log).

9.4 Notes on the bridge script

The REST bridge (`ai_pentest_bridge.py`) has three settings: the Ollama endpoint (the Windows hostname on :11434), the model name (pentest-assistant), and the HexStrike endpoint (`http://127.0.0.1:8888/api/command`). Its loop is: ask the model for one command, show it for approval, run it through HexStrike, feed the output back, repeat. It is the simplest honest way to show a local model driving HexStrike, and it does not use MCP.

10. Risks and Safeguards Sheet

One page summary. This sheet answers the four questions from the assignment.

What are the risks?

- The AI agent has powerful system access and can run real attack tools. In our own test it launched a Metasploit exploit module and brute forced a real SSH login, so a wrong or unreviewed command could do real damage.
- Pointing the tool at the wrong target, on purpose or by mistake, would be illegal and unauthorized.
- A deliberately vulnerable target (Metasploitable) could be reached by others if it is not isolated.
- The HexStrike API has no authentication, so anyone who can reach :8888 can run tools.
- The local model can be unreliable at tool calling, which can produce wrong or incomplete actions if nobody is watching.

What controls should be implemented?

- A human in the loop approval on every command (we kept this on the whole time).
- A strict authorized target list, and never testing anything outside it.
- Host only network isolation for vulnerable machines, with no internet access.
- Tool allow listing, so the model can only call the tools the job needs.
- Authentication on the HexStrike API and keeping :8888 local only.
- Full audit logging from the model, the client and the server, and pinned model and library versions.

What level of human oversight is required?

High. A qualified person must approve each tool call, must confirm the target is authorized before anything runs, and must read the results rather than trusting the model blindly. The AI speeds up the work and chains the tools, but it does not replace the operator who is responsible for the test. In our setup the model never ran anything without us approving it first.

What are the legal, ethical and operational limitations?

- **Legal.** Running these tools against any system without written authorization is illegal. In this lab we only used a public practice target (megacorpone) and a machine we own (Metasploitable).
- **Ethical.** The findings and target data are sensitive. Keeping everything on a local model is partly an ethical choice, so nothing leaks to a third party.
- **Operational.** The local model is limited by the hardware and is weaker at structured tool calling than a cloud model, the setup is a lab build and not a hardened production deployment, and the whole stack has to be kept patched and isolated by the operator.